Gatling API Tool - Fresh Runbook

Version: fresh baseline • Coverage: startup, launch, business use cases, full-feature dummy config, expert mode, reporting, release packaging

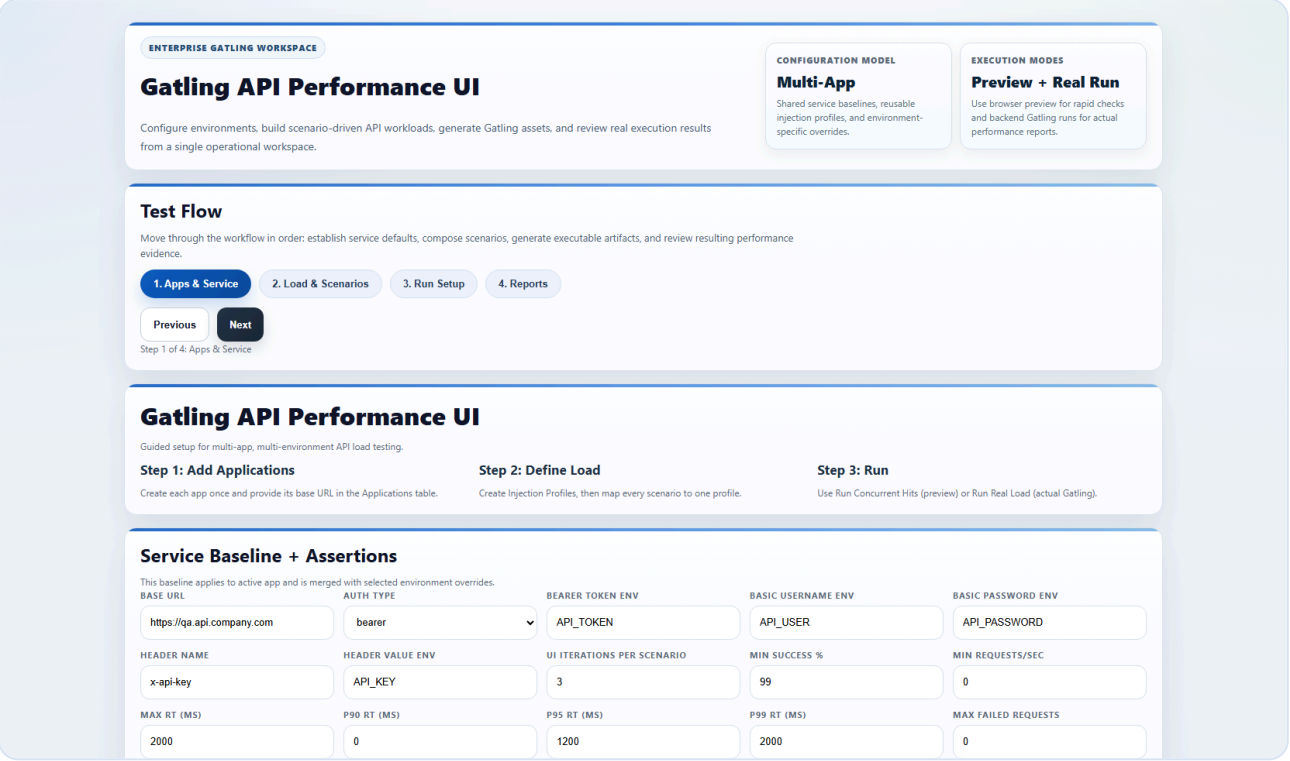
1) Business Objectives and Use Cases

Primary Business Value

- Prevent release regressions by validating response-time and success-rate gates before production deployment.
- Provide a repeatable QA baseline with saved suites and environment-aware configs.
- Accelerate incident triage through real run diagnostics and report drill-downs.

Business Use Cases

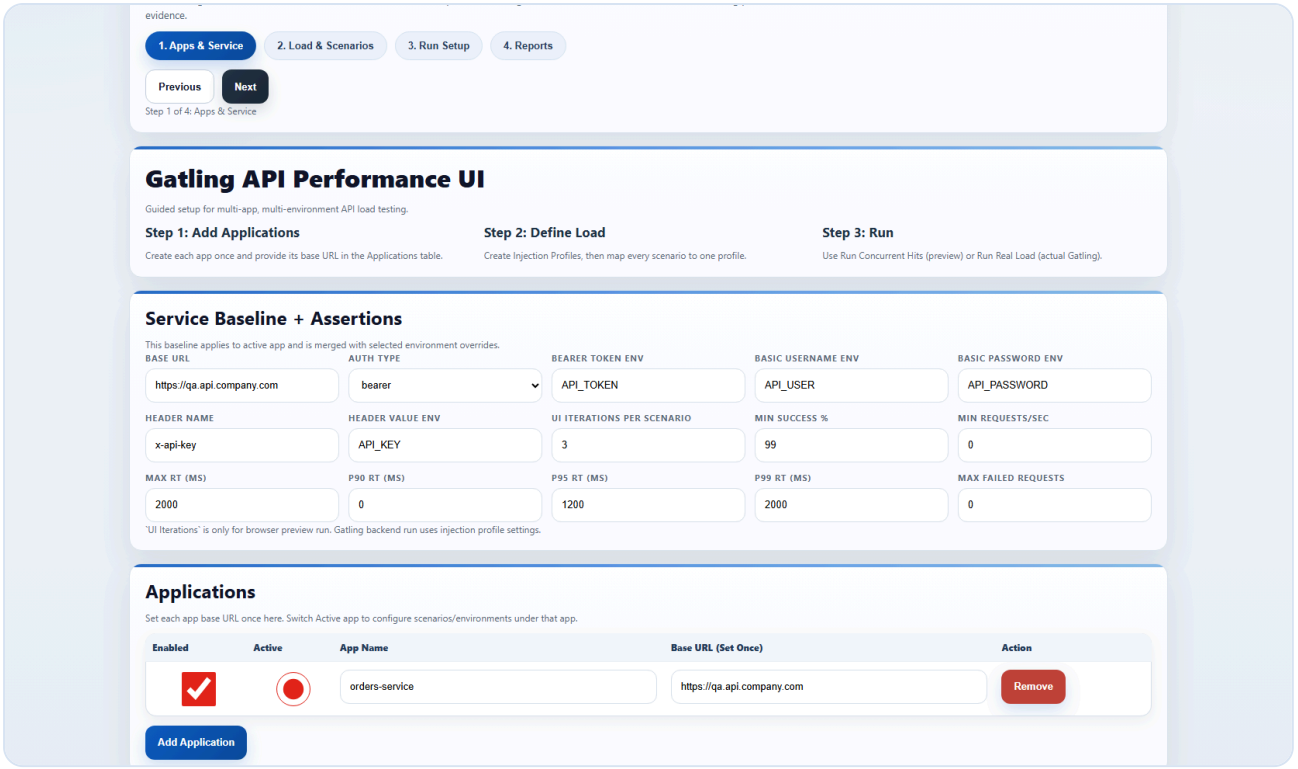
1. Pre-release performance gate for core order APIs.
2. Daily regression run for QA environment.
3. Partner onboarding load validation (header/basic auth variants).
4. Capacity planning via smoke/baseline/stress profiles.
5. Post-incident replay with raw YAML override for exact reproduction.

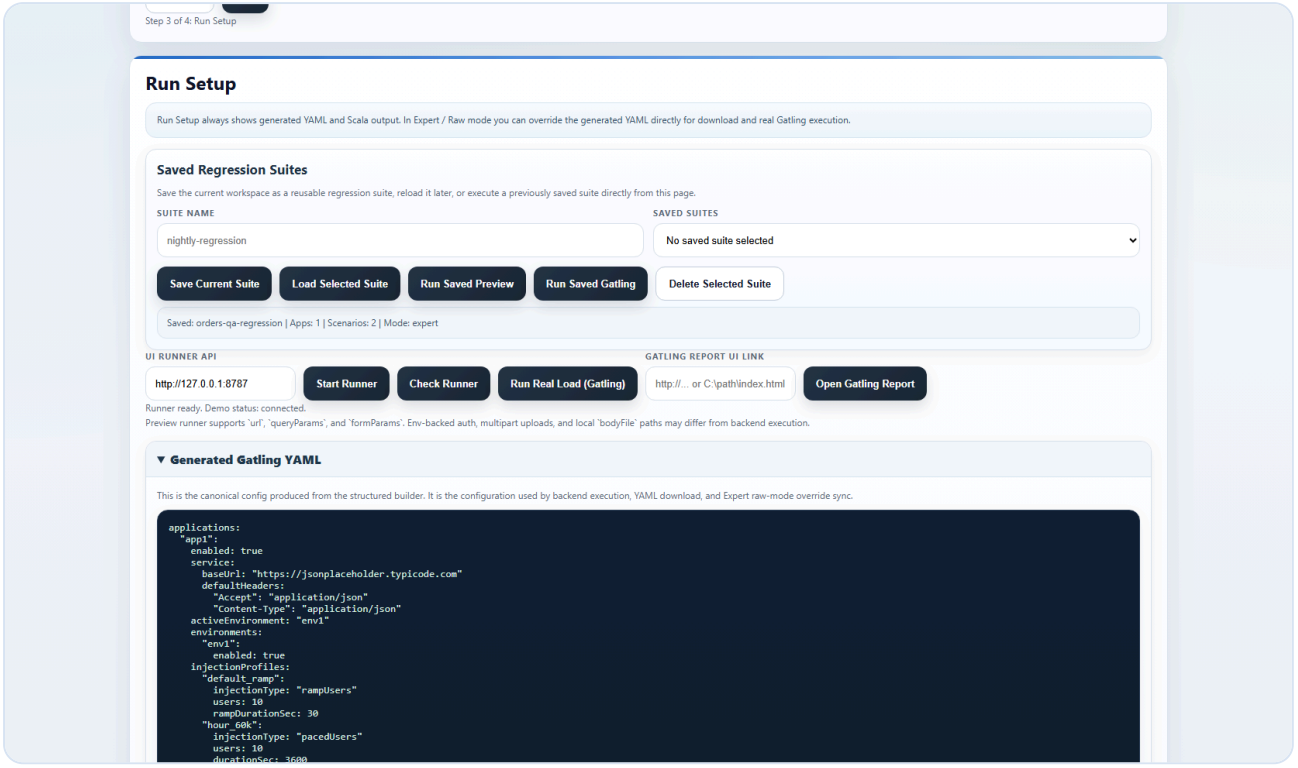


2) Startup and Launch

1. Open terminal in `gatling-api-tool`.
2. Run prerequisite checks.
3. Start workspace (backend + UI).
4. Verify runner endpoint in Run Setup.

```
cd gatling-api-tool
check-prerequisites.bat
start-ui-workspace.bat
```


evidence.
1. Apps & Service
2. Load & Scenarios
3. Run Setup
4. Reports
Previous
Next
Step 1 of 4: Apps & Service
Gatling API Performance UI
Guided setup for multi-app, multi-environment API load testing.
Step 1: Add Applications
Create each app once and provide its base URL in the Applications table.
Step 2: Define Load
Create Injection Profiles, then map every scenario to one profile.
Step 3: Run
Use Run Concurrent Hits (preview) or Run Real Load (actual Gatling).
Service Baseline + Assertions
This baseline applies to active app and is merged with selected environment overrides.
BASE URL
https://qa.api.company.com
AUTH TYPE
bearer
BEARER TOKEN ENV
API_TOKEN
BASIC USERNAME ENV
API_USER
BASIC PASSWORD ENV
API_PASSWORD
HEADER NAME
x-api-key
HEADER VALUE ENV
API_KEY
UI ITERATIONS PER SCENARIO
3
MIN SUCCESS %
99
MIN REQUESTS/SEC
0
MAX RT (MS)
2000
P90 RT (MS)
0
P95 RT (MS)
1200
P99 RT (MS)
2000
MAX FAILED REQUESTS
0
`UI Iterations` is only for browser preview run. Gatling backend run uses injection profile settings.
Applications
Set each app base URL once here. Switch Active app to configure scenarios/environments under that app.
Enabled
Active
App Name
Base URL (Set Once)
Action
orders-service
https://qa.api.company.com
Remove
Add Application


Step 3 of 4: Run Setup
Run Setup
Run Setup always shows generated YAML and Scala output. In Expert / Raw mode you can override the generated YAML directly for download and real Gatling execution.
Saved Regression Suites
Save the current workspace as a reusable regression suite, reload it later, or execute a previously saved suite directly from this page.
SUITE NAME
nightly-regression
SAVED SUITES
No saved suite selected
Save Current Suite
Load Selected Suite
Run Saved Preview
Run Saved Gatling
Delete Selected Suite
Saved: orders-qa-regression | Apps: 1 | Scenarios: 2 | Mode: expert
UI RUNNER API
http://127.0.0.1:8787
Start Runner
Check Runner
Run Real Load (Gatling)
GATLING REPORT UI LINK
http://... or C:\path\index.html
Open Gatling Report
Runner ready. Demo status: connected.
Preview runner supports `url`, `queryParams`, and `formParams`. Env-backed auth, multipart uploads, and local `bodyFile` paths may differ from backend execution.
▼ Generated Gatling YAML
This is the canonical config produced from the structured builder. It is the configuration used by backend execution, YAML download, and Expert raw-mode override sync.
applications:
"app1":
enabled: true
service:
baseUrl: "https://jsonplaceholder.typicode.com"
defaultHeaders:
"Accept": "application/json"
"Content-Type": "application/json"
activeEnvironment: "env1"
environments:
"env1":
enabled: true
injectionProfiles:
"default_ramp":
injectionType: "rampUsers"
users: 10
rampDurationSec: 30
"hour_60k":
injectionType: "pacedUsers"
users: 10

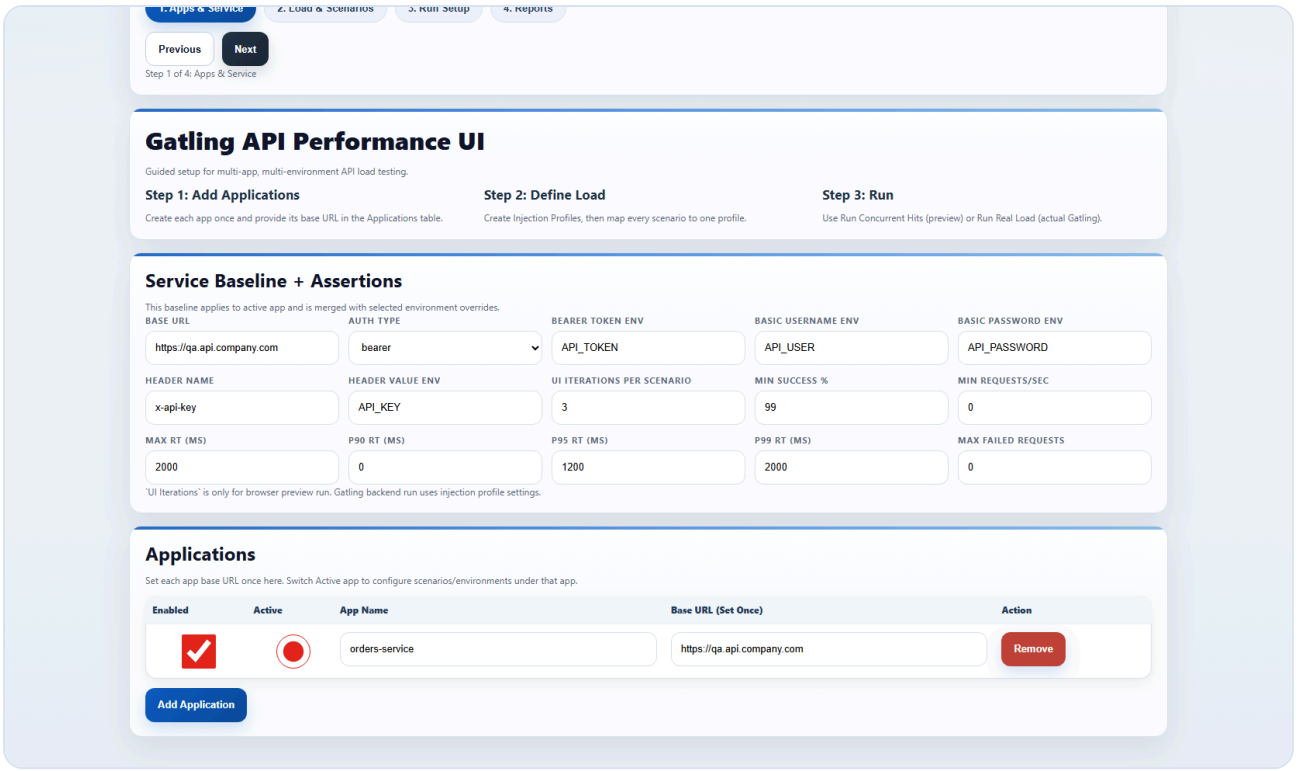
3) Dummy Scenario Definition (orders-service)

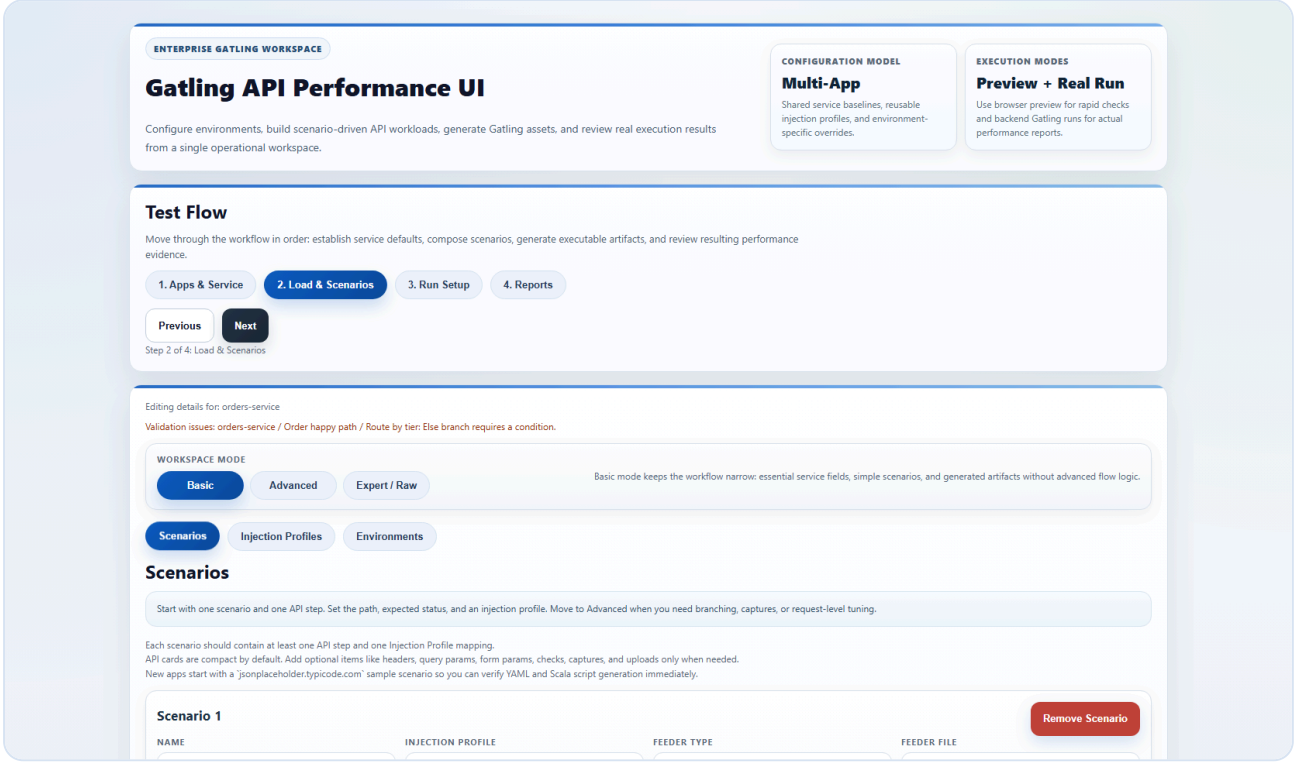
Flow

- `GET /orders` -> capture `orderId` and `customerTier`
- Conditional branch: if premium -> `/orders/premium/#{orderId}` else -> `/orders/standard/#{orderId}`
- `POST /orders` with JSON payload
- Optional multipart import scenario

SLO Targets

- Success >= 99%
- p95 <= 1200ms
- p99 <= 2000ms
- Max failed requests = 0

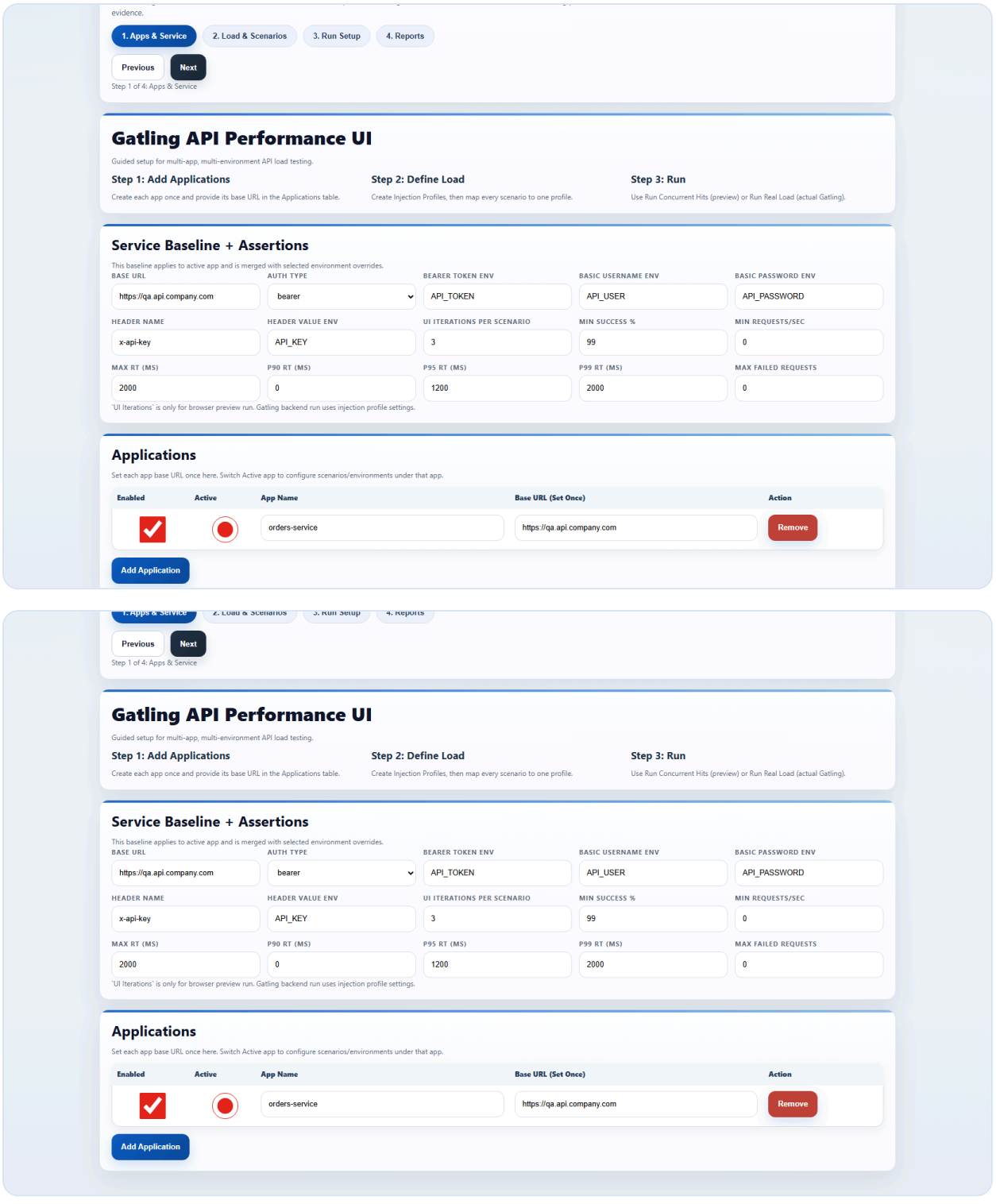

1. Apps & Service
2. Load & Scenarios
3. Run Setup
4. Reports
Previous
Next
Step 1 of 4: Apps & Service
Gatling API Performance UI
Guided setup for multi-app, multi-environment API load testing.
Step 1: Add Applications
Create each app once and provide its base URL in the Applications table.
Step 2: Define Load
Create Injection Profiles, then map every scenario to one profile.
Step 3: Run
Use Run Concurrent Hits (preview) or Run Real Load (actual Gatling).
Service Baseline + Assertions
This baseline applies to active app and is merged with selected environment overrides.
BASE URL
https://qa.api.company.com
AUTH TYPE
bearer
BEARER TOKEN ENV
API_TOKEN
BASIC USERNAME ENV
API_USER
BASIC PASSWORD ENV
API_PASSWORD
HEADER NAME
x-api-key
HEADER VALUE ENV
API_KEY
UI ITERATIONS PER SCENARIO
3
MIN SUCCESS %
99
MIN REQUESTS/SEC
0
MAX RT (MS)
2000
P90 RT (MS)
0
P95 RT (MS)
1200
P99 RT (MS)
2000
MAX FAILED REQUESTS
0
'UI Iterations' is only for browser preview run. Gatling backend run uses injection profile settings.
Applications
Set each app base URL once here. Switch Active app to configure scenarios/environments under that app.
Enabled
Active
App Name
Base URL (Set Once)
Action
orders-service
https://qa.api.company.com
Remove
Add Application


ENTERPRISE GATLING WORKSPACE
Gatling API Performance UI
Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.
CONFIGURATION MODEL
Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.
EXECUTION MODES
Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.
Test Flow
Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.
1. Apps & Service
2. Load & Scenarios
3. Run Setup
4. Reports
Previous
Next
Step 2 of 4: Load & Scenarios
Editing details for: orders-service
Validation issues: orders-service / Order happy path / Route by tier: Else branch requires a condition.
WORKSPACE MODE
Basic
Advanced
Expert / Raw
Basic mode keeps the workflow narrow: essential service fields, simple scenarios, and generated artifacts without advanced flow logic.
Scenarios
Injection Profiles
Environments
Scenarios
Start with one scenario and one API step. Set the path, expected status, and an injection profile. Move to Advanced when you need branching, captures, or request-level tuning.
Each scenario should contain at least one API step and one Injection Profile mapping.
API cards are compact by default. Add optional items like headers, query params, form params, checks, captures, and uploads only when needed.
New apps start with a `jsonplaceholder.typicode.com` sample scenario so you can verify YAML and Scala script generation immediately.
Scenario 1
Remove Scenario
NAME
INJECTION PROFILE
FEEDER TYPE
FEEDER FILE

4) Feature-by-Feature Configuration Walkthrough

Service, Assertions, and App Setup

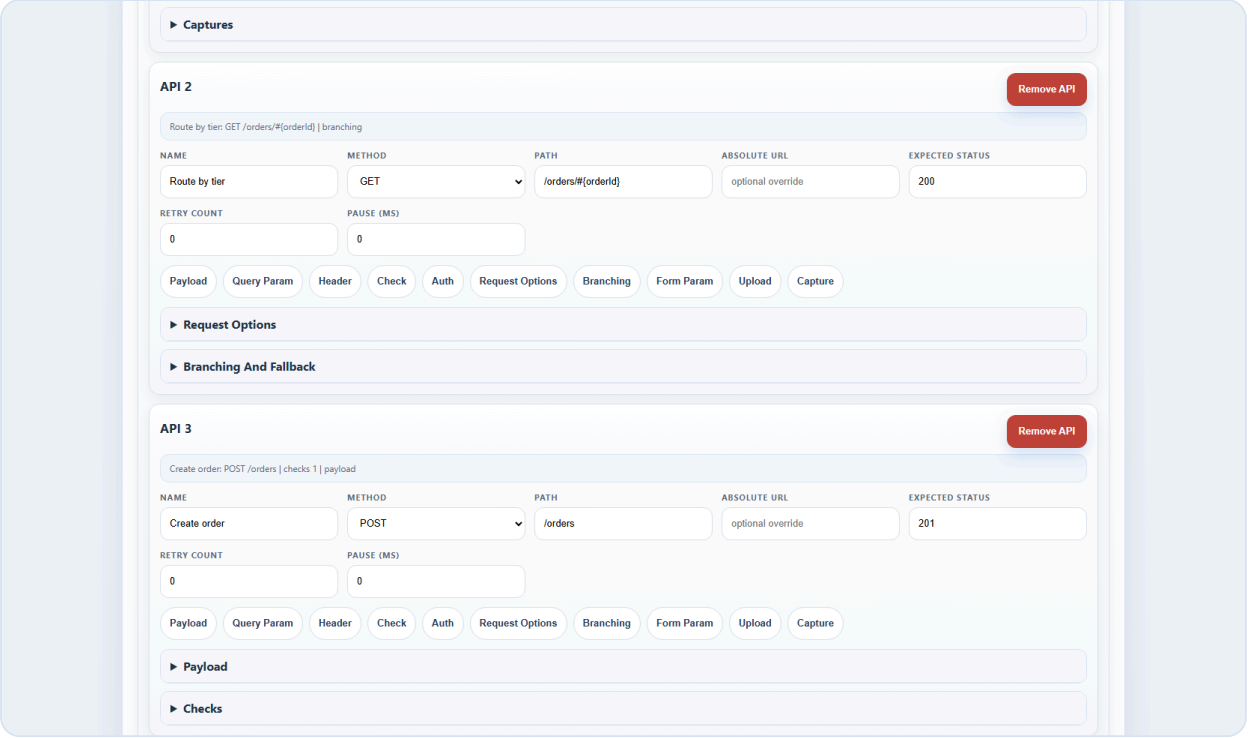
- Set base URL, auth type, assertion thresholds.
- Add/activate app entry in Applications table.

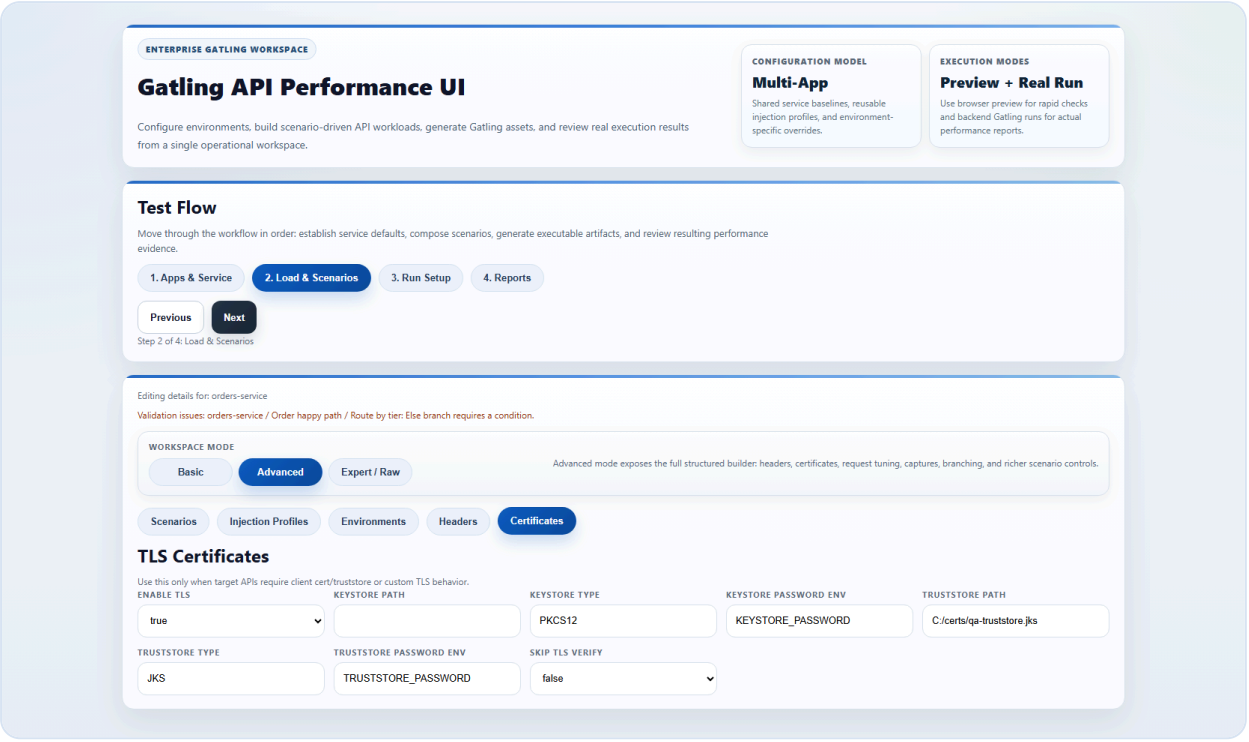


Scenarios, Checks, Captures, Conditional Branches

- Define steps and expected statuses.
- Add checks and captures to drive data-dependent flow.

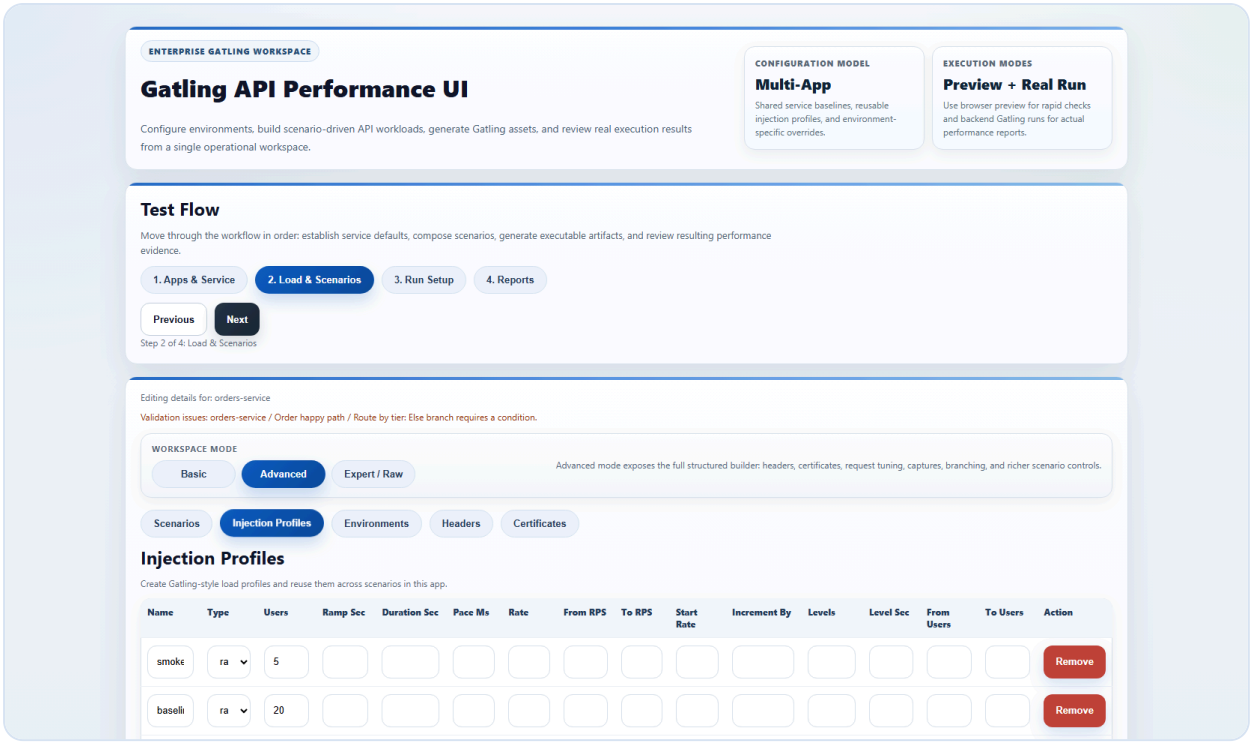
- Configure branch/else behavior and request flags.

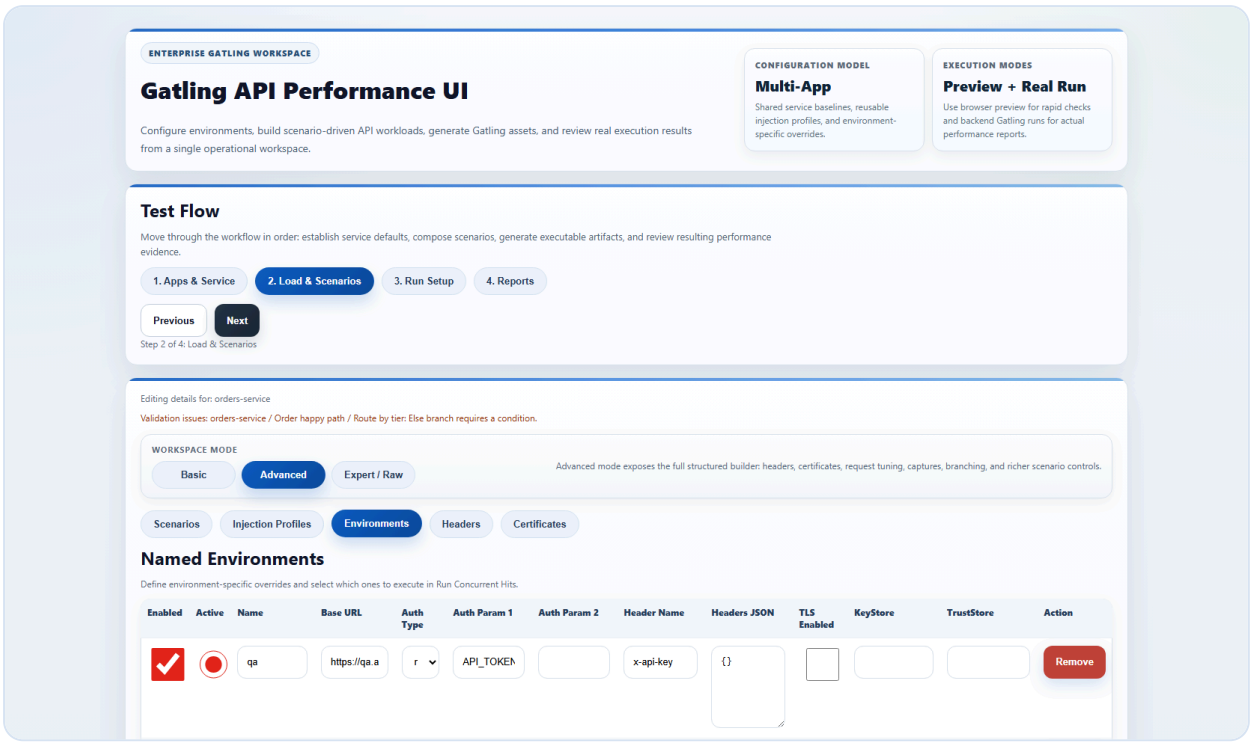


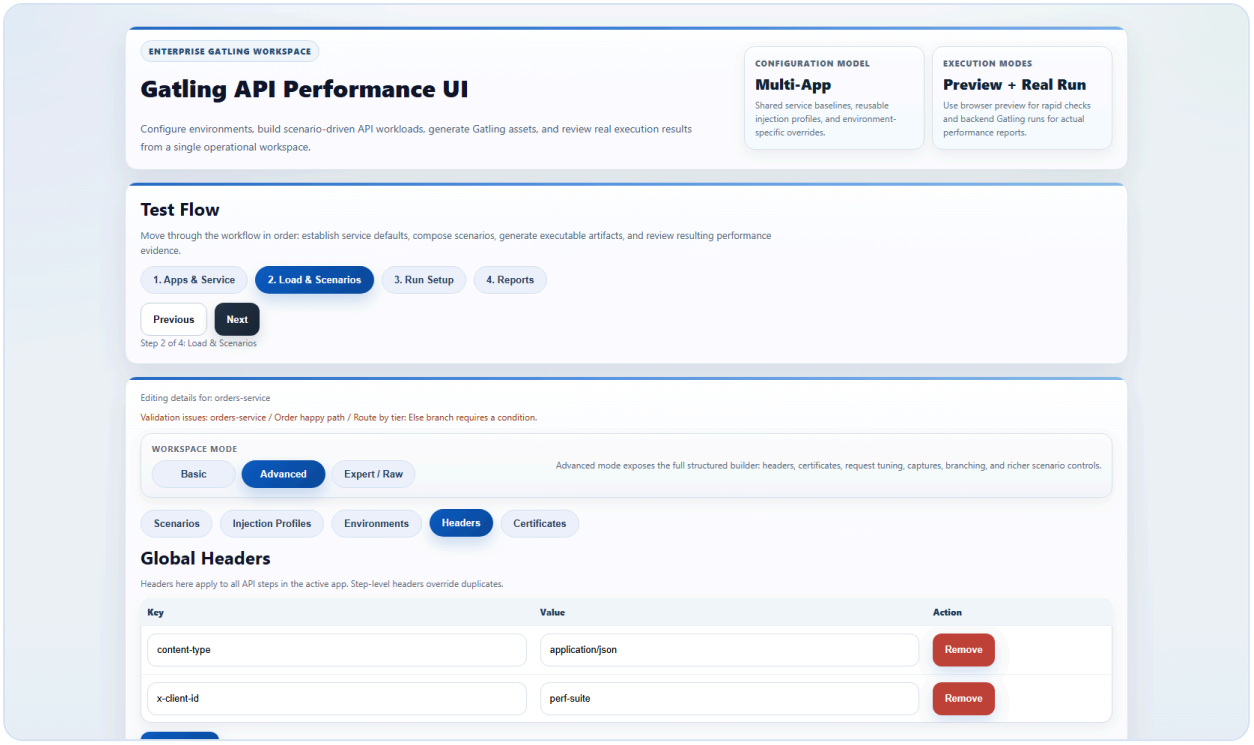


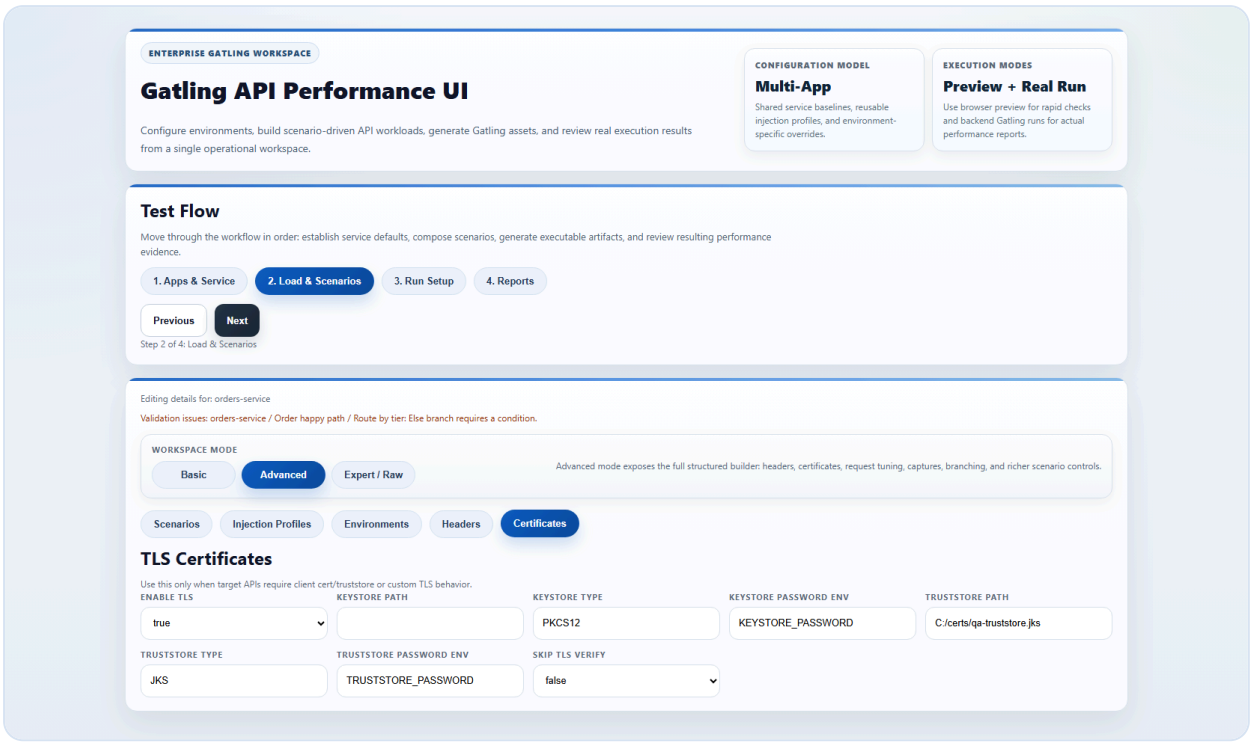
Injection Profiles, Environments, Headers, TLS

- Create smoke/baseline/stress profiles and map to scenarios.
- Add QA/prod-like environments and auth overrides.
- Set global headers and optional TLS cert/truststore values.


ENTERPRISE GATLING WORKSPACE
Gatling API Performance UI
Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.
CONFIGURATION MODEL
Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.
EXECUTION MODES
Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.
Test Flow
Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.
1. Apps & Service
2. Load & Scenarios
3. Run Setup
4. Reports
Previous
Next
Step 2 of 4: Load & Scenarios
Editing details for: orders-service
Validation issues: orders-service / Order happy path / Route by tier: Else branch requires a condition.
WORKSPACE MODE
Basic
Advanced
Expert / Raw
Advanced mode exposes the full structured builder: headers, certificates, request tuning, captures, branching, and richer scenario controls.
Scenarios
Injection Profiles
Environments
Headers
Certificates
Injection Profiles
Create Gatling-style load profiles and reuse them across scenarios in this app.
Name
Type
Users
Ramp Sec
Duration Sec
Pace Ms
Rate
From RPS
To RPS
Start Rate
Increment By
Levels
Level Sec
From Users
To Users
Action
smoke
ra
5
Remove
baselli
ra
20
Remove


ENTERPRISE GATLING WORKSPACE
Gatling API Performance UI
Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.
CONFIGURATION MODEL
Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.
EXECUTION MODES
Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.
Test Flow
Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.
1. Apps & Service
2. Load & Scenarios
3. Run Setup
4. Reports
Previous
Next
Step 2 of 4: Load & Scenarios
Editing details for: orders-service
Validation issues: orders-service / Order happy path / Route by tier: Else branch requires a condition.
WORKSPACE MODE
Basic
Advanced
Expert / Raw
Advanced mode exposes the full structured builder: headers, certificates, request tuning, captures, branching, and richer scenario controls.
Scenarios
Injection Profiles
Environments
Headers
Certificates
Named Environments
Define environment-specific overrides and select which ones to execute in Run Concurrent Hits.
Enabled
Active
Name
Base URL
Auth Type
Auth Param 1
Auth Param 2
Header Name
Headers JSON
TLS Enabled
KeyStore
TrustStore
Action
qa
https://qa.a
r
API_TOKEN
x-api-key
{}
Remove





Run Setup, Raw YAML, Reports, Saved Suites

- Use generated YAML and expert raw YAML override as needed.
- Run real load, inspect diagnostics, and validate report KPIs.
- Save suite for reusable regression execution.

This is the canonical config produced from the structured builder. It is the configuration used by backend execution, YAML download, and Expert raw-mode override sync.

```yaml
applications:
  "app1":
    enabled: true
    service:
      baseUrl: "https://jsonplaceholder.typicode.com"
      defaultHeaders:
        "Accept": "application/json"
        "Content-Type": "application/json"
    activeEnvironment: "env1"
    environments:
      "env1":
        enabled: true
    injectionProfiles:
      "default_ramp":
        injectionType: "rampUsers"
        users: 10
        rampDurationSec: 30
      "hour_60k":
        injectionType: "pacedUsers"
        users: 10
        durationSec: 3600
        paceMs: 600
    assertions:
      minSuccessPercent: 99
      maxResponseTimeMs: 2000
      p95ResponseTimeMs: 1200
    scenarios:
      - name: "Read Posts"
        load:
          injectionType: "rampUsers"
          profileRef: "default_ramp"
          users: 10
          rampDurationSec: 30
        steps:
          - name: "Get Posts"
            method: "GET"
            path: "/posts"
            expectedStatus: 200
            retryCount: 0
            pauseMs: 0
            queryParams:
              "_limit": "10"
            checks:
              - type: "jsonPathExists"
                path: "$[0].id"
          - name: "Get Post By Id"
            method: "GET"
            path: "/posts/1"
            expectedStatus: 200
            retryCount: 0
            pauseMs: 0
            checks:
              - type: "jsonPathEquals"
                path: "$.id"
                value: "1"
```

▼ **Generated Scala Gatling Script**

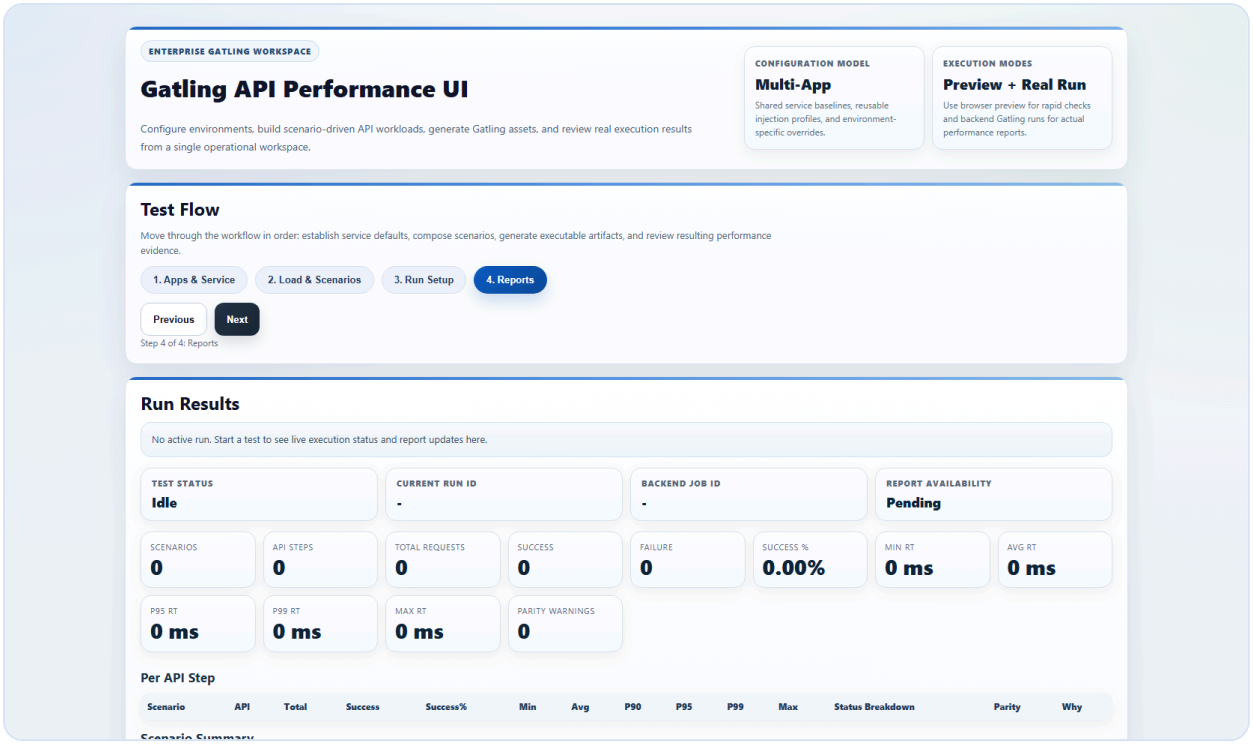
```scala
    var scn = scenario("app1 :: Read Posts")
    scn = scn.exec(app1_Read_Posts_chain)
    scn
  }

  setUp(
       app1_Read_Posts_scenario.injectOpen(rampUsers(10) during (30.seconds)).protocols(http_app1)
  )
    .assertions(
      details("app1 :: Read Posts").successfulRequests.percent.gte(99),
      details("app1 :: Read Posts").responseTime.max.lte(2000),
      details("app1 :: Read Posts").responseTime.percentile3.lte(1200)
    )

  private def requireEnv(name: String): String = {
    val value = sys.env.getOrElse(name, "")
    require(value.nonEmpty, s"Missing env var: $name")
    value
  }

  private def basicAuthHeader(userEnv: String, passEnv: String): String = {
    val raw = s"${requireEnv(userEnv)}:${requireEnv(passEnv)}"
    "Basic " + Base64.getEncoder.encodeToString(raw.getBytes(StandardCharsets.UTF_8))
  }

  private def readFile(path: String): String = Files.readString(Paths.get(path))
}
```

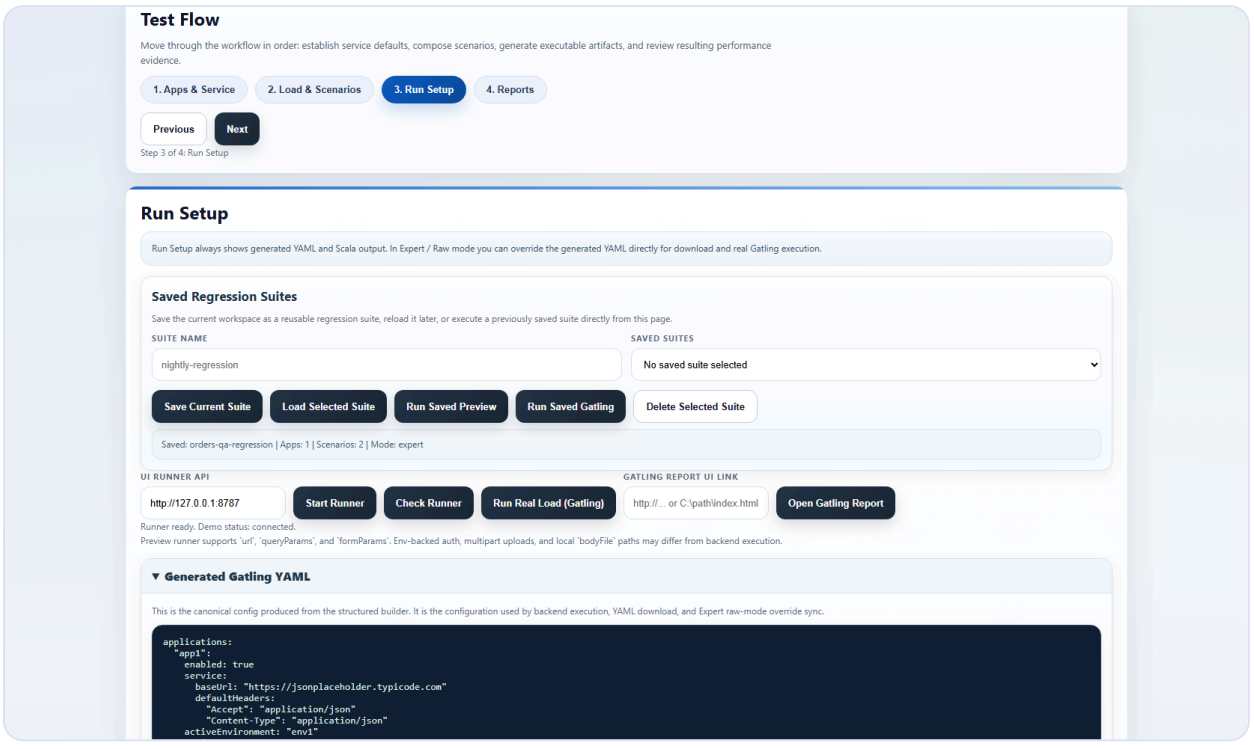
Expert / Raw YAML Override

Use this only when you need a full-fidelity YAML escape hatch beyond the structured builder. When enabled, real runs and YAML download use this editor content directly. Scala script generation still comes from the structured UI model.

Sync Generated YAML Into Editor | Use Raw YAML: Off

```yaml
applications:
  "orders-service":
    enabled: true
    service:
      baseUrl: "https://qa.api.company.com"
      defaultHeaders:
        "content-type": "application/json"
        "x-client-id": "perf-suite"
      auth:
        type: "bearer"
        tokenEnv: "API_TOKEN"
      tls:
        enabled: true
        keyStoreType: "PKCS12"
        keyStorePasswordEnv: "KEYSTORE_PASSWORD"
        trustStorePath: "C:/certs/qa-truststore.jks"
        trustStoreType: "JKS"
```


ENTERPRISE GATLING WORKSPACE
Gatling API Performance UI
Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.
CONFIGURATION MODEL
Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.
EXECUTION MODES
Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.
Test Flow
Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.
1. Apps & Service
2. Load & Scenarios
3. Run Setup
4. Reports
Previous
Next
Step 4 of 4: Reports
Run Results
No active run. Start a test to see live execution status and report updates here.
TEST STATUS
Idle
CURRENT RUN ID
-
BACKEND JOB ID
-
REPORT AVAILABILITY
Pending
SCENARIOS
0
API STEPS
0
TOTAL REQUESTS
0
SUCCESS
0
FAILURE
0
SUCCESS %
0.00%
MIN RT
0 ms
AVG RT
0 ms
P95 RT
0 ms
P99 RT
0 ms
MAX RT
0 ms
PARITY WARNINGS
0
Per API Step
Scenario
API
Total
Success
Success%
Min
Avg
P90
P95
P99
Max
Status Breakdown
Parity
Why


Test Flow
Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.
1. Apps & Service
2. Load & Scenarios
3. Run Setup
4. Reports
Previous
Next
Step 3 of 4: Run Setup
Run Setup
Run Setup always shows generated YAML and Scala output. In Expert / Raw mode you can override the generated YAML directly for download and real Gatling execution.
Saved Regression Suites
Save the current workspace as a reusable regression suite, reload it later, or execute a previously saved suite directly from this page.
SUITE NAME
nightly-regression
SAVED SUITES
No saved suite selected
Save Current Suite
Load Selected Suite
Run Saved Preview
Run Saved Gatling
Delete Selected Suite
Saved: orders-qa-regression | Apps: 1 | Scenarios: 2 | Mode: expert
UI RUNNER API
http://127.0.0.1:8787
Start Runner
Check Runner
Run Real Load (Gatling)
GATLING REPORT UI LINK
http://... or C:\path\index.html
Open Gatling Report
Runner ready. Demo status: connected.
Preview runner supports 'url', 'queryParams', and 'formParams'. Env-backed auth, multipart uploads, and local 'bodyFile' paths may differ from backend execution.
Generated Gatling YAML
This is the canonical config produced from the structured builder. It is the configuration used by backend execution, YAML download, and Expert raw-mode override sync.
applications:
"app1":
enabled: true
service:
baseUrl: "https://jsonplaceholder.typicode.com"
defaultHeaders:
"Accept": "application/json"
"Content-Type": "application/json"
activeEnvironment: "env1"

5) Full Dummy Config (All Major Features)

File generated at: `dist/dummy-full-feature-config.yaml`

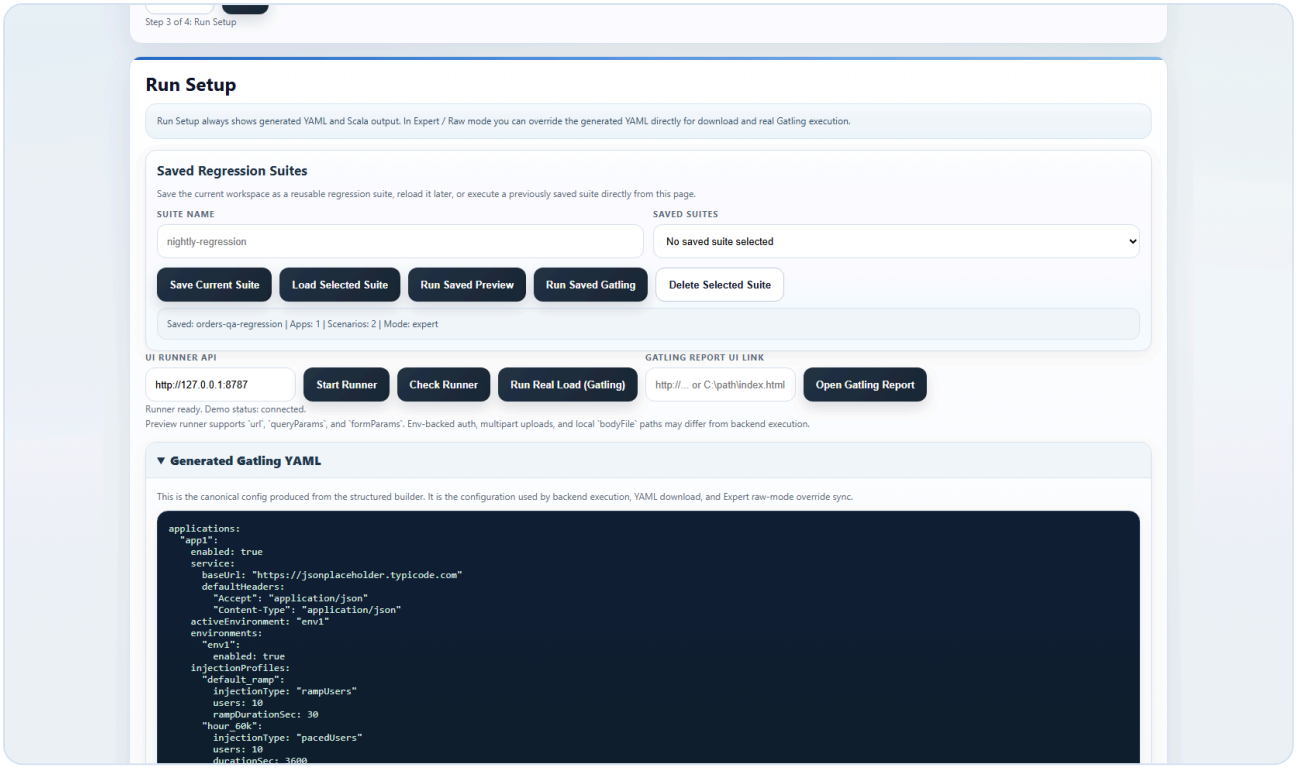
```yaml
applications:
  orders-service:
    enabled: true
    service:
      baseUrl: "https://qa.api.company.com"
      defaultHeaders:
        content-type: "application/json"
        x-client-id: "perf-suite"
      auth:
        type: bearer
        tokenEnv: API_TOKEN
      tls:
        enabled: true
        trustStorePath: "C:/certs/qa-truststore.jks"
        trustStoreType: JKS
        trustStorePasswordEnv: TRUSTSTORE_PASSWORD
        insecureSkipTlsVerify: false
    assertions:
      minSuccessPercent: 99
      maxResponseTimeMs: 2000
      p95ResponseTimeMs: 1200
      p99ResponseTimeMs: 2000
      maxFailedRequests: 0
    environments:
      qa:
        enabled: true
        baseUrl: "https://qa.api.company.com"
      prod-like:
        enabled: false
        baseUrl: "https://prodlike.api.company.com"
        auth:
          type: header
          headerName: "x-api-key"
          headerValueEnv: "PRODLIKE_API_KEY"
    injectionProfiles:
      smoke_5:
        injectionType: rampUsers
        users: 5
        rampSec: 15
      baseline_20:
        injectionType: rampUsers
        users: 20
        rampSec: 60
      stress_80:
        injectionType: rampUsers
        users: 80
        rampSec: 180
    scenarios:
      - name: "Order happy path"
        enabled: true
        feeder:
          type: csv
          file: "src/test/resources/data/users.csv"
          mode: circular
```

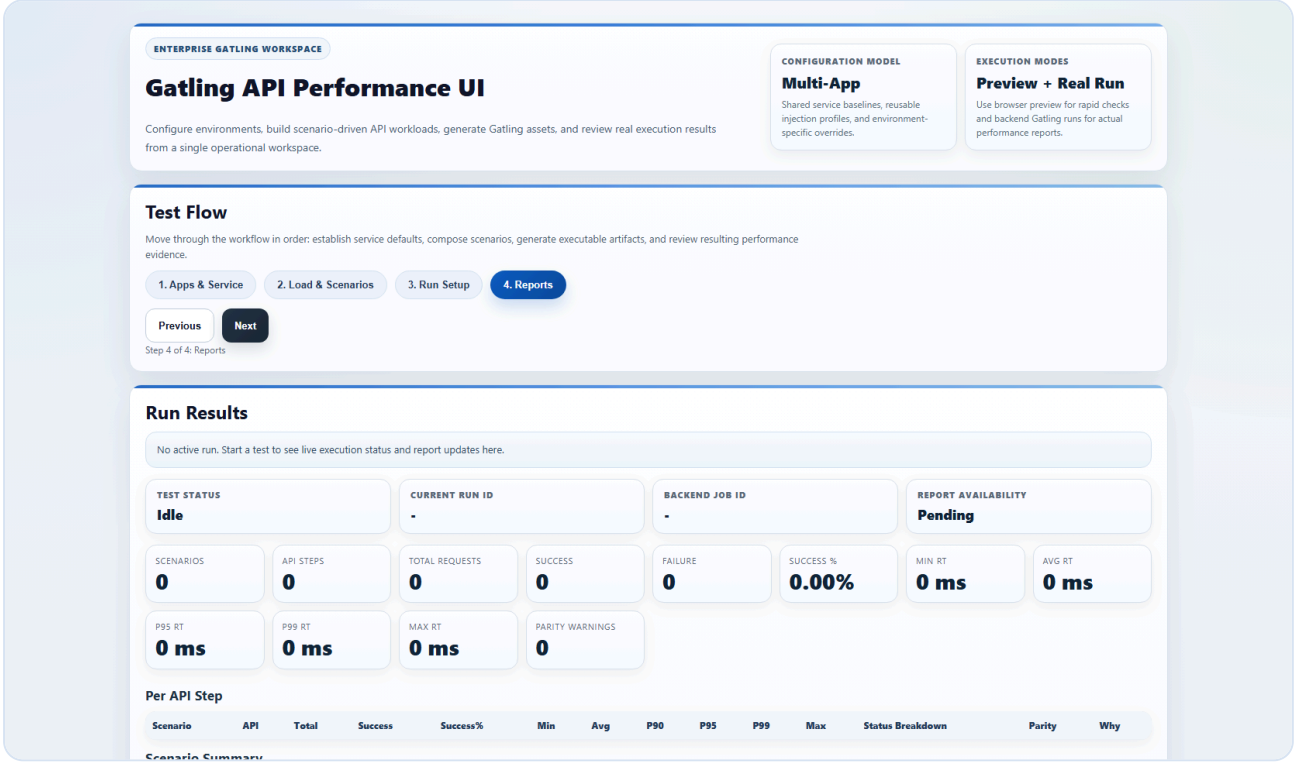
```
    load:
      profileRef: baseline_20
    flow:
      exitOnFail: true
    steps:
      - name: "List orders"
        method: GET
        path: "/orders"
        expectedStatus: 200
        queryParams:
          customerId: "#{customerId}"
          page: "1"
        checks:
          - type: jsonPathExists
            path: "$.items[0].id"
        captures:
          - jsonPath: "$.items[0].id"
            saveAs: "orderId"
          - jsonPath: "$.items[0].customerTier"
            saveAs: "customerTier"
      - name: "Route by tier"
        method: GET
        path: "/orders/#{orderId}"
        expectedStatus: 200
        branches:
          - name: "Premium route"
            when:
              variable: customerTier
              operator: equals
              value: premium
            method: GET
            path: "/orders/premium/#{orderId}"
            expectedStatus: 200
        elseMethod: GET
        elsePath: "/orders/standard/#{orderId}"
        elseExpectedStatus: 200
        requestTimeoutMs: 30000
        disableFollowRedirect: true
      - name: "Create order"
        method: POST
        path: "/orders"
        bodyFile: "src/test/resources/bodies/create-order.json"
        bodyType: json
        expectedStatus: 201
        checks:
          - type: bodyContains
            value: "created"
  - name: "Import orders"
    enabled: true
    load:
      profileRef: smoke_5
    steps:
      - name: "Upload order file"
        method: POST
        path: "/orders/import"
        bodyType: multipart
        expectedStatus: 202
        formParams:
          source: "qa"
        formUploads:
```

```
              - fieldName: "file"
                filePath: "src/test/resources/data/upload-sample.txt"
```

6) Real Run Procedure

1. Go to Run Setup page.
2. Click **Check Runner**.
3. Click **Run Real Load (Gatling)**.
4. Monitor status tiles (job ID, state, report availability).
5. Analyze report tables and diagnostics.





7) Packaging, Handoff, and Governance

- Use `build-release.bat` to run checks + bundle + checksum.
- Distribute zip plus `latest-bundle.sha256` for integrity verification.
- Track runbook and dummy config as versioned project artifacts.

```
build-release.bat
```

Generated runbook from local project context and captured UI screenshots.